DIGITAL QUEUE MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

YASWANTH SREENIVAS D

(240701611)

in partial fulfilment of the End Semester Examination for the course

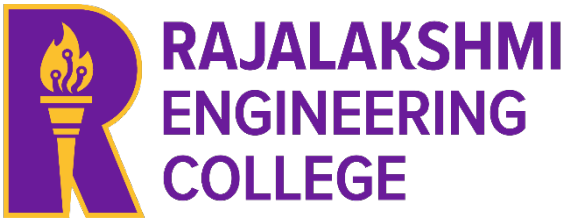
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

Computer Science Engineering



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

2026 - 2027

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled “Digital Queue Management System” is the bonafide work of YASWANTH SREENIVAS D (240701611), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. J. Manoranjini
HEAD OF THE DEPARTMENT
Department of CSE
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Mr. Pravin P
COURSE FACULTY
Department of CSE
Rajalakshmi Engineering College
Thandalam – 602 105

Submitted for the End Semester Examination (Individual Project) held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, YASWANTH SREENIVAS D (240701611), hereby declare that the project report titled “Digital Queue Management System”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place:_____ **SIGNATURE OF THE STUDENT**

Date:_____ YASWANTH SREENIVAS D

240701611

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Shankar Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S.N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. J. Manoranjini, Head of the Department, Department of CSE, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Mr. Pravin P, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

YASWANTH SREENIVAS D

ABSTRACT

Service centres, administrative offices, banks and campus helpdesks handle many visitor enquiries and service requests every day across multiple counters. When visitor queues and token allocations are recorded manually in paper registers or spreadsheets, it becomes difficult to know how many slots remain at each counter, which tokens are currently being served and when pending tokens are scheduled. This can lead to duplicate visitor records, tokens issued beyond counter capacity, long physical waiting lines and manual calculation of late rescheduling fees.

This project presents the Digital Queue Management System, a low-code application developed on Zoho Creator to manage visitor registration, counter capacity, token generation and service completion in one place. The application has five forms – Visitor, Service_Counter, Token_Request, Queue_Service and Feedback – connected through lookup fields, eight reports and an Admin Dashboard that shows summary figures and a chart of counters by service category. Deluge scripts validate visitor IDs, token requests and counter calls, update the available counter slots automatically when tokens are called and completed, and calculate the rescheduling fee at ₹10 per delayed day. A scheduled workflow runs daily to mark delayed queue transactions and notify the visitor, and automated emails inform the administrator and visitors about token requests, approvals, active counter calls and delayed tokens.

The application is managed through the administrator account of Zoho Creator. Separate visitor login roles and a customer portal are not implemented because the current plan requires an upgrade, so visitor activities are demonstrated through sample records. The application demonstrates counter slot tracking, token validation, administrative approval, queue calling and completion management, delay monitoring, fee calculation and email-based communication in a centralised digital queue management system.

Keywords: ***Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Queue Management; Token Scheduling***

PROJECT AT A GLANCE

Attribute	Details
Project Title	Digital Queue Management System
Application Domain	Service Operations – Digital Queue and Token Management
Student Name & Register No.	YASWANTH SREENIVAS D – 240701611
Zoho Creator Edition	FREE
Application Name (Link Name)	DIGITAL QUEUE MANAGEMENT SYSTEM
Published / Live URL	https://creatorapp.zoho.in/ yaswanth.sreenivas.2024.cse_rajala/digital-queue-management-system
Demo Credentials (per role)	Administrator – root / root
Number of Forms / Subforms	5 forms / 0 subforms
Number of Reports (by type)	List: 8 | Kanban: 0 | Calendar: 0 | Pivot/Chart: 0 | Others: 0
Number of Pages / Dashboards	1 (Admin Dashboard)
Number of Workflows	Form: 9 | Schedule: 1 | Approval: 1 | Blueprint: 1 | Batch: 0
Deluge Functions / Custom Actions	9 Deluge scripts in form and scheduled workflows; no separate functions or custom actions
Integrations	No external integrations; automated email notifications through Deluge sendmail
Roles & Profiles Defined	Administrator (existing administrator account only)
Environments Used	Development (single environment)
Date of Production Deployment	14-10-2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY 4
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 4
2.3 Low-Code versus Traditional Development 5
2.4 Literature Survey 5
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 6
REQUIREMENT ANALYSIS 8
3.1 Stakeholders and User Roles 8
3.2 Functional Requirements 8
3.3 Non-Functional Requirements 10
3.4 Hardware and Software Requirements 10
3.5 Use Case Model 10
SYSTEM DESIGN 13
4.1 System Architecture 13
4.2 Data Model 13
4.3 Form Design 14
4.4 Process and Workflow Design 16
4.5 User Interface and Navigation Design 17
4.6 Access Control Design 17
IMPLEMENTATION 19
5.1 Application Setup 19
5.2 Forms and Data Capture 19

5.3 Reports 20
5.4 Pages and Dashboards 21
5.5 Deluge Scripting 22
5.6 Workflows and Automation 24
5.7 Integrations 26
5.8 User Management and Role-Based Access 26
5.9 AI and Analytics Features 26
5.10 Pro-Code Extension 27
5.11 Mobile and Portal Access 27
TESTING AND DEBUGGING 28
6.1 Testing Strategy 28
6.2 Test Cases 28
6.3 Debugging Deluge Scripts 29
6.4 Defect Log 30
6.5 User Acceptance Testing 30
DEPLOYMENT, SECURITY AND GOVERNANCE 31
7.1 Environments and Deployment Strategy 31
7.2 Performance Optimisation 31
7.3 Security Features 31
7.4 Governance and Administration 32
RESULTS AND DISCUSSION 33
8.1 Application Output 33
8.2 Achievement of Objectives 34
8.3 Comparison with the Existing System 34
8.4 Limitations of the Low-Code Approach 35
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 35
CONCLUSION AND FUTURE ENHANCEMENTS 36
9.1 Conclusion 36
9.2 Future Enhancements 36
9.3 Learning Outcomes and Career Relevance 36
REFERENCES 37
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 38
APPENDIX B – COURSE OUTCOME MAPPING 41
APPENDIX C – COMPLETE DELUGE SOURCE CODE 42
APPENDIX D – USER MANUAL 46
D.1 Accessing the Application 46
D.2 Instructions for Administrator 46

D.3 Instructions for Visitor
APPENDIX E – EVALUATION RUBRIC 48
APPENDIX F – VIVA-VOCE RECORD SHEET 50

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Table 2.2: Summary of Literature Survey

Table 3.1: Stakeholders and User Roles

Table 3.2: Functional Requirements

Table 3.3: Non-Functional Requirements

Table 3.4: Hardware and Software Requirements

Table 3.5: Use Case Description – Submit and Approve a Token Request

Table 3.6: Use Case Description – Call Token and Complete Service

Table 4.1: Entities and Relationships

Table 4.2: Field Specification – Visitor Form

Table 4.3: Field Specification – Service_Counter Form

Table 4.4: Field Specification – Token_Request Form

Table 4.5: Field Specification – Queue_Service Form

Table 4.6: Field Specification – Feedback Form

Table 4.7: Role–Permission Matrix

Table 5.1: Reports Implemented

Table 5.2: Deluge Scripts Implemented

Table 5.3: Workflows Implemented

Table 5.4: Integrations Implemented

Table 6.1: Test Cases

Table 6.2: Defect Log

Table 6.3: User Acceptance Feedback

Table 7.1: Deployment Checklist

Table 7.2: Security Controls Implemented

Table 8.1: Mapping of Objectives to Outcomes

Table 8.2: Existing System versus Proposed Application

Table A.1: Feature Coverage Checklist

Table B.1: Mapping of Course Outcomes to Report Chapters

Table E.1: Evaluation Rubric for the Individual Project

Table E.2: Performance Level Descriptors

Table F.1: Viva-Voce Questions and Responses

Table F.2: Marks Summary

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing Manual Queue Process
Figure 3.1: Use Case Diagram of the Digital Queue Management System
Figure 4.1: System Architecture of the Digital Queue Management System
Figure 4.2: Entity–Relationship Diagram of the Application
Figure 4.3: Workflow / Process Flow of the Queue Management Process
Figure 4.4: Navigation Structure and Wireframe of the Admin Dashboard
Figure 5.1: Application Home and Navigation Sections
Figure 5.2: Token_Request Form in the Form Builder
Figure 5.3: Token_Request Form in Live Mode
Figure 5.4: Available Counters Report
Figure 5.5: Admin Dashboard
Figure 5.6: Workflow Configuration – Visitor ID Validation
Figure 8.1: Scenario Step 1 – Token Request Submitted
Figure 8.2: Scenario Step 2 – Token Approved and Called to Counter
Figure 8.3: Scenario Step 3 – Service Completed and Fee Calculated

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
DS	Deluge Script
ER	Entity–Relationship
FR	Functional Requirement
ID	Identifier
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho's AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Administrative offices, banks, hospitals and campus service centres manage high daily visitor traffic across specialised service counters such as fee collection, certificate issuance and scholarship helpdesks. Because each service counter has a finite daily capacity and fixed operating hours, the institution must keep track of how many counters are active, how many token slots remain available, who has requested a token and when each visitor is scheduled to be served.

Two parties are involved in this process. Visitors request queue tokens for the counter service they need, and the administrator maintains the counter capacities, approves or rejects token requests, calls tokens to counters, records service completions and applies late rescheduling fees for delayed appointments. The Digital Queue Management System, built on Zoho Creator, brings these activities into a single application where visitor details, counter slots, token requests, active service transactions, delayed records and visitor feedback are maintained together.

1.2 Problem Statement

Managing visitor queues and service counters manually is time-consuming and error-prone. The administrator has to check remaining counter slots by hand before approving a token request, track which visitors are waiting or delayed and calculate fees for late appointments manually. The institution needs an application that checks token requests against actual counter capacity, keeps available slots updated automatically and notifies visitors about delayed tokens. The main pain points are:

- Visitor and counter records can be duplicated or inconsistent when kept in separate paper registers or spreadsheets.
- Token requests can be approved or called for more slots than are actually available at a service counter.
- The available slot count has to be updated by hand whenever tokens are called to a counter or completed.
- Delayed or unserved appointments are noticed late and rescheduling fees are calculated manually.

- The administrator has no single place to view pending token requests, active called tokens and delayed tokens.

1.3 Objectives

- To design a data model and forms for managing visitors, service counters, token requests, queue service transactions, and feedback.
- To automate token request validation, slot capacity updates, service completion processing, fee calculation and delayed token checking using Deluge scripting and workflows.
- To provide reports and an Admin Dashboard for monitoring counters, token requests, active called tokens and delayed tokens.
- To send automated email notifications to the administrator and visitors at the main stages of the queue management process.
- To demonstrate the complete queue management lifecycle – token request, approval, counter call, service completion, delay handling and feedback – on the Zoho Creator platform.

1.4 Scope of the Project

In scope: The application manages visitor records and service counter capacities, submission and validation of token requests, approval or rejection by the administrator, calling and completing tokens with automatic update of available slots, late fee calculation at ₹10 per delayed day, a daily delayed-queue check, feedback collection, eight reports and an Admin Dashboard. The application is used on the Zoho Creator web platform through the administrator account, which is the only user role.

Out of scope: Separate visitor login roles and a self-service customer portal are not implemented because the current Zoho Creator plan requires an upgrade for the access features needed. Visitor activities are therefore demonstrated through sample records managed within the application. Hardware token display boards, SMS gateways and online payment of fees are also not part of this project.

1.5 Justification for a Low-Code Approach

Low-code platforms allow business applications to be built mainly by configuring forms, reports and workflows visually, with a small amount of scripting where custom

logic is needed [1], [2]. Zoho Creator is suitable for this project for the following reasons. Forms, lookup relationships and reports are created visually, which suits the structured nature of a queue management system. Deluge scripting [3] provides the slot validation and fee calculation logic that cannot be achieved through configuration alone. Scheduled workflows and email notifications are built into the platform, so the daily delayed-token check and the notifications need no separate server. The application is also hosted by the platform, so no infrastructure had to be set up. As a result, the complete system could be built and maintained without conventional programming.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In many offices and campus service centres, visitor queues are handled manually. A visitor approaches the helpdesk or counter staff for a token, and the staff member checks the counter's daily capacity in a paper register or spreadsheet and writes down the token number by hand along with the date and the visitor's details. When the visitor is served, the register entry is marked completed and the slot count is updated manually. Delayed or missed appointments are noticed only when the register is inspected, and any rescheduling fee is calculated by hand.

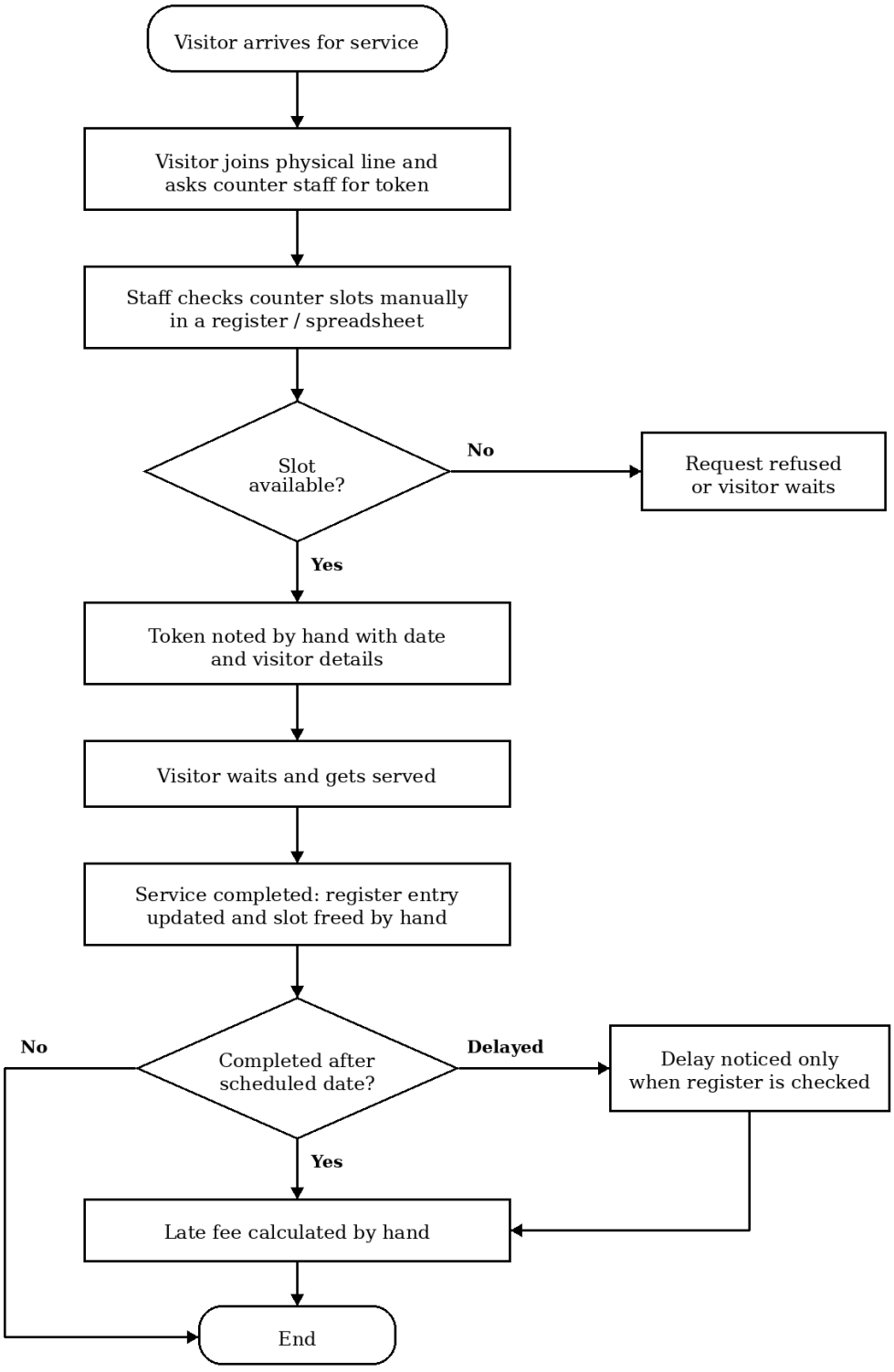


Figure 2.1: As-Is Process Flow of the Existing Manual Queue Process

2.2 Limitations of the Existing System

- Counter slot availability has to be checked manually, so tokens may be promised or issued beyond the daily capacity of the counter.
- Records are scattered across registers and spreadsheets, so duplicate visitor entries or inconsistent token numbers are likely.
- Delayed tokens and rescheduling fees depend on manual checking and calculation.
- Visitors and the administrator receive no automatic notifications about token requests, approvals, counter calls or scheduled dates.
- Lists of waiting, called, served and delayed tokens have to be prepared by hand.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Longer, because the database, back end and user interface must be developed and tested separately.	Shorter, because forms, reports and workflows are built visually and can be tested immediately.
Skills required	Programming in a web language, database design and server administration.	Understanding of forms, lookup relationships and basic Deluge scripting.
Hosting & infrastructure	Server and database must be set up and maintained by the college.	Hosted and maintained by the Zoho Creator platform.
Mobile application	A separate mobile application or responsive design has to be developed.	Provided by the platform; not configured in this project.
Security & access control	Login, roles and permissions must be coded.	Provided by the platform; only the administrator account is used in this project.
Maintenance & change	Code changes, testing and redeployment are needed for every change.	Fields, reports and workflows can be edited in the builder.
Customisation limits	Complete freedom in design and logic.	Limited to the features of Zoho Creator and Deluge.
Cost	Development effort, hosting and maintenance costs.	Platform subscription with lower development effort; the current plan limits access features.

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	A. C. Bock and U. Frank, 2021 [4]	Low-Code Platform	Describes low-code platforms as environments in which applications are built through visual, model-based development with little hand-written code.	Explains the visual approach used for the forms, reports and workflows of this application.
2	A. Sahay et al., 2020 [5]	Supporting the Understanding and Comparison of Low-Code Development Platforms	Compares low-code development platforms on the basis of their features and capabilities.	Helps in understanding the typical features, such as forms, workflows and scripting, offered by Zoho Creator.
3	Y. Luo et al., 2021 [6]	Characteristics and Challenges of Low-Code Development: The Practitioners' Perspective	Reports practitioners' views on the benefits of low-code development, such as faster development, and on its challenges, such as platform limits.	Relates to the benefits and limitations discussed in Sections 1.5 and 8.4.
4	Zoho Corporation [2]	Zoho Creator Help Documentation	Explains forms, reports, pages, workflows and administration in Zoho Creator.	Used as the main reference for building the application.
5	Zoho Corporation [3]	Deluge Script Reference Guide	Explains Deluge syntax, fetch statements, conditions, loops and email tasks.	Used for the validation and automation scripts.

2.5 Emerging Trends in Low-Code Development

Current trends in low-code development include AI-assisted application building, development by non-programmers (citizen development) and the automation of routine service workflows. In this project, the emphasis was on automating routine queue management tasks through form workflows and a scheduled process so that the

application can be maintained by an administrator without heavy coding. AI-assisted features such as Zia were not used in this application.

2.6 Proposed System

The proposed system is the Digital Queue Management System built on Zoho Creator. All visitor, service counter, token request and queue service data is stored in related forms, which removes scattered paper registers. Validation scripts check the Visitor_ID, the requested slots, the validity dates and the available counter capacity before records are saved, which prevents duplicate visitors and over-booking. Counter call and service completion workflows update the available slots automatically. A daily scheduled workflow marks delayed transactions, calculates the rescheduling fee and emails the visitor, and email notifications keep the administrator and visitors informed. Reports and the Admin Dashboard give the administrator a single view of counter capacities and queue activity.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Person in charge of the service counters and queue operations, using the existing Zoho Creator administrator account.	Maintain visitor and service counter records, review token requests, approve or reject requests, manage counter call and completion transactions, and view reports and the dashboard.
Visitor	Person requesting service at a counter. Has no separate login in the current application.	Requests queue tokens, receives email notifications about approval, counter call and delayed status, and gives feedback. These activities are demonstrated through sample records managed by the administrator.

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall store visitor details and shall reject a new visitor whose Visitor_ID already exists.	Administrator	High

FR ID	Requirement	Role(s)	Priority
FR-02	The system shall store service counter details including maximum daily capacity, available slots, counter status, location, queue permission and standard service duration.	Administrator	High
FR-03	The system shall validate a token request: the counter must exist, the tickets requested must be greater than zero and not exceed the available slots, and Valid_Till_Date must not be earlier than Preferred_Date.	Administrator	High
FR-04	The system shall email the administrator when a new token request is submitted.	Administrator	Medium
FR-05	The system shall email the visitor when the administrator changes the token status to Approved or Rejected, and only when the status changes.	Administrator	High
FR-06	The system shall validate a queue service call: a valid token request must be selected, the slots allocated must be greater than zero and the counter must have sufficient available slots.	Administrator	High
FR-07	The system shall reduce the available slots and email the visitor after a token is called to a counter.	Administrator	High
FR-08	The system shall set the status to Served and restore the allocated slots to the counter capacity only once when a service transaction is completed.	Administrator	High
FR-09	The system shall calculate the rescheduling fee at ₹10 per delayed day for late completions and set it to zero for on-time completions.	Administrator	High
FR-10	The system shall run a daily check that changes unserved Called transactions past their scheduled date to Delayed, updates the fee and emails the visitor.	Administrator	High
FR-11	The system shall record feedback with visitor, service counter, rating (1 to 5), comments and date.	Administrator	Medium
FR-12	The system shall provide reports for visitors, available counters, pending token requests, approved token requests, active called tokens, delayed tokens, served tokens and visitor queue history.	Administrator	Medium
FR-13	The system shall provide an Admin Dashboard showing total counters, available slots, called tokens, pending requests and delayed tokens and a chart of counters by service category.	Administrator	Medium

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports shall show only the records that match their filter criteria so that the administrator sees the relevant queue records quickly.
NFR-02	Security	Only the administrator account shall be able to view and modify the application data.
NFR-03	Usability	The application shall be used through the Zoho Creator web interface, with alerts that explain validation errors.
NFR-04	Availability	The application shall be hosted on the Zoho Creator cloud platform and be accessible through a browser with an internet connection.
NFR-05	Maintainability	Forms, reports and workflows shall be built with the visual builder and Deluge so that they can be modified easily.
NFR-06	Auditability	Every queue service transaction shall be recorded with its dates, priority, resolution status, fee and remarks.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (cloud-based low-code platform)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
API testing tool	Not used
Other Zoho services	None. Email notifications are sent using the built-in sendmail task of Deluge.
Mobile	Not used
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

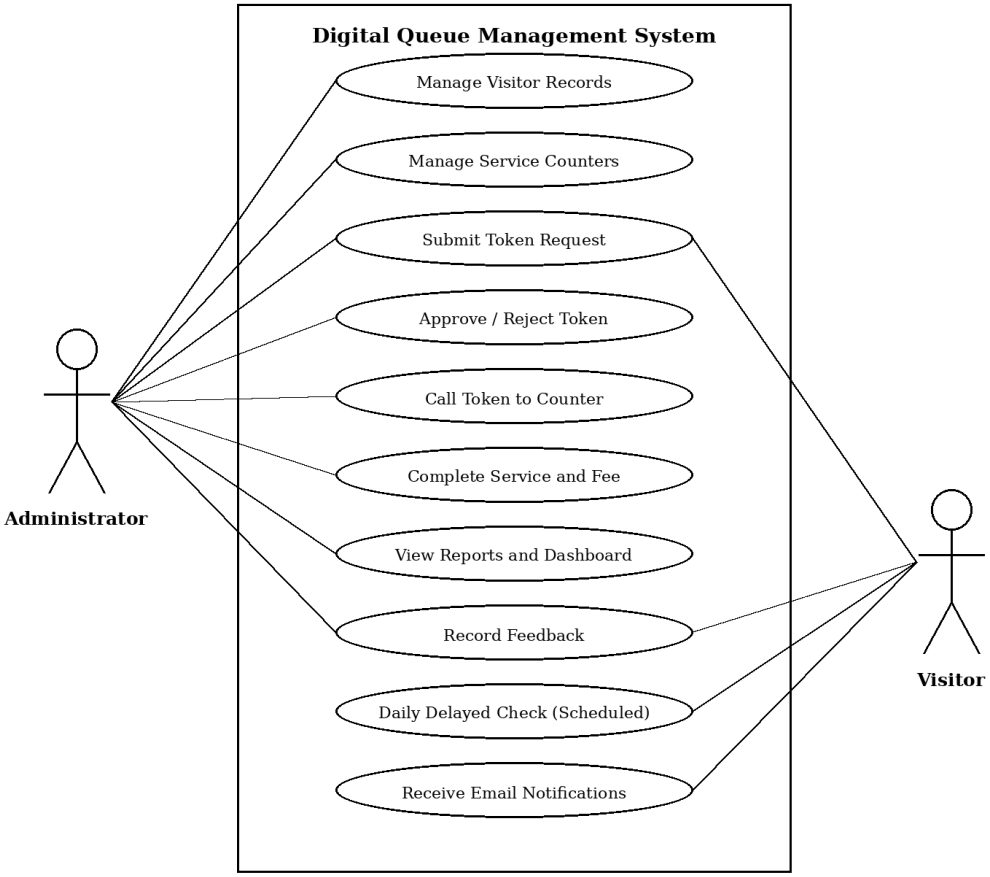


Figure 3.1: Use Case Diagram of the Digital Queue Management System

Table 3.5: Use Case Description – Submit and Approve a Token Request

Use Case ID	UC-01
Use Case Name	Submit and Approve a Token Request
Actor(s)	Administrator (visitor details are held in the Visitor form; the visitor receives email notifications)
Pre-condition	Visitor and service counter records exist, and the counter has available slots for booking.

Main Flow	1. The administrator opens the Token_Request form and selects the visitor and the service counter. 2. The administrator enters the tickets requested, booking and preferred dates and service purpose. 3. The system validates the request and saves it. 4. The system emails the administrator about the new token request. 5. The administrator reviews the request in the Pending Token Requests report and sets Token_Status to Approved or Rejected. 6. The system emails the visitor about the decision.
Alternate Flow	If the service counter is missing, the tickets requested is zero or more than the available slots, or Valid_Till_Date is earlier than Preferred_Date, an alert is shown and the request is not saved.
Post-condition	The token request is saved with the status Approved or Rejected and the visitor is notified.
Related FR	FR-03, FR-04, FR-05

Table 3.6: Use Case Description – Call Token and Complete Service

Use Case ID	UC-02
Use Case Name	Call Token and Complete Service
Actor(s)	Administrator (the visitor receives email notifications)
Pre-condition	An approved token request exists and the service counter has sufficient available slots.
Main Flow	1. The administrator opens the Queue_Service form and selects the Token_Request. 2. The administrator enters the slots allocated, call date, scheduled service date and priority at call. 3. The system validates the entry, saves it, reduces the Available_Slots and emails the visitor. 4. When the visitor is served, the administrator records Actual_Completion_Date and Resolution_Status. 5. The system sets the status to Served, calculates the fee and restores the counter slot once.
Alternate Flow	If the visitor is not served by the scheduled date, the daily scheduled workflow changes the status to Delayed, updates the fee and emails the visitor. For no-show or cancelled tokens the administrator records the status and adjusts the counter slots accordingly.
Post-condition	The queue service transaction is closed and the counter slot capacity is updated.
Related FR	FR-06, FR-07, FR-08, FR-09, FR-10

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

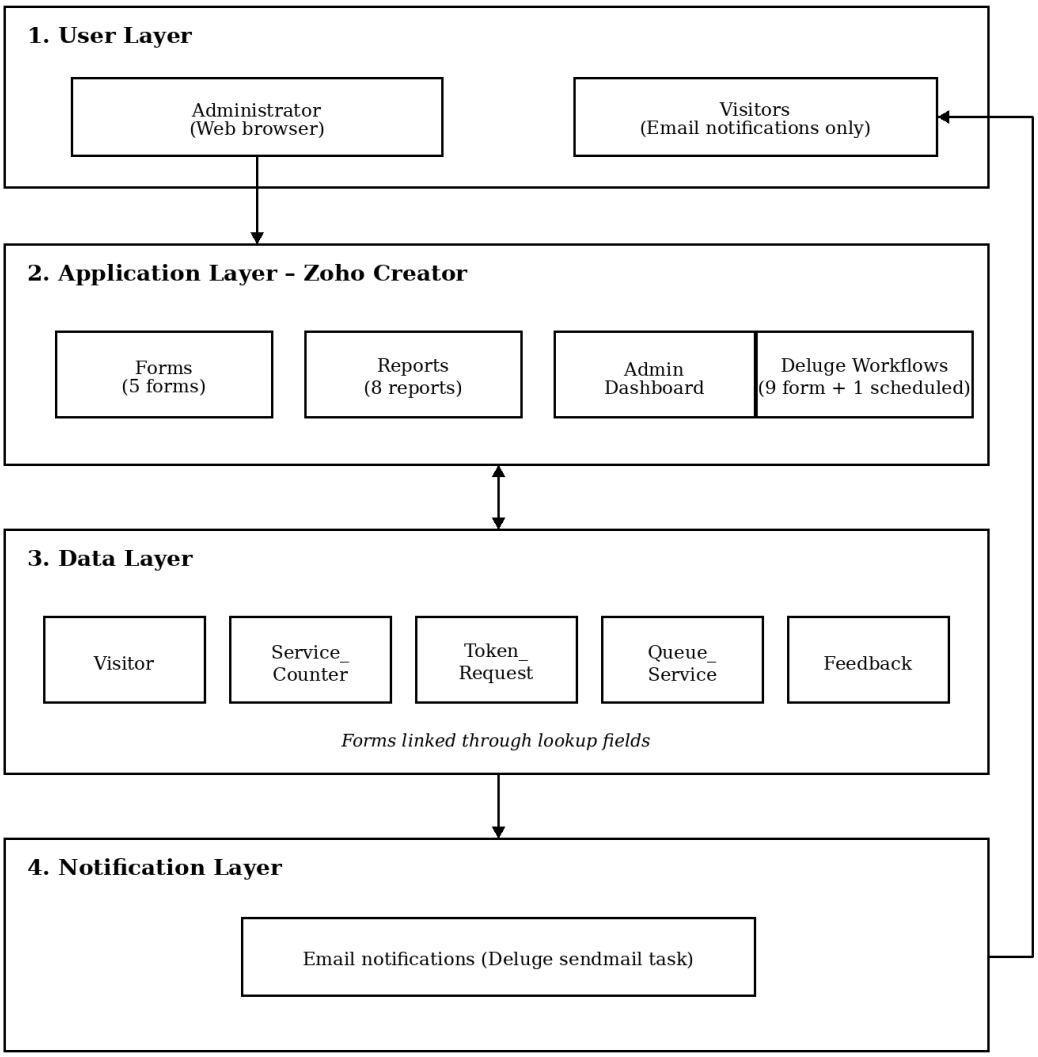


Figure 4.1: System Architecture of the Digital Queue Management System

The application has four layers. At the user layer, the administrator uses a web browser to access the Zoho Creator application, while visitors are informed through email notifications. At the application layer, Zoho Creator provides the five forms, the reports, the Admin Dashboard and the Deluge workflows (form workflows and one scheduled workflow). At the data layer, the form records are stored in Zoho Creator and linked through lookup fields. At the notification layer, the Deluge sendmail task

sends emails to the administrator and to visitors. No external system is connected to the application.

4.2 Data Model

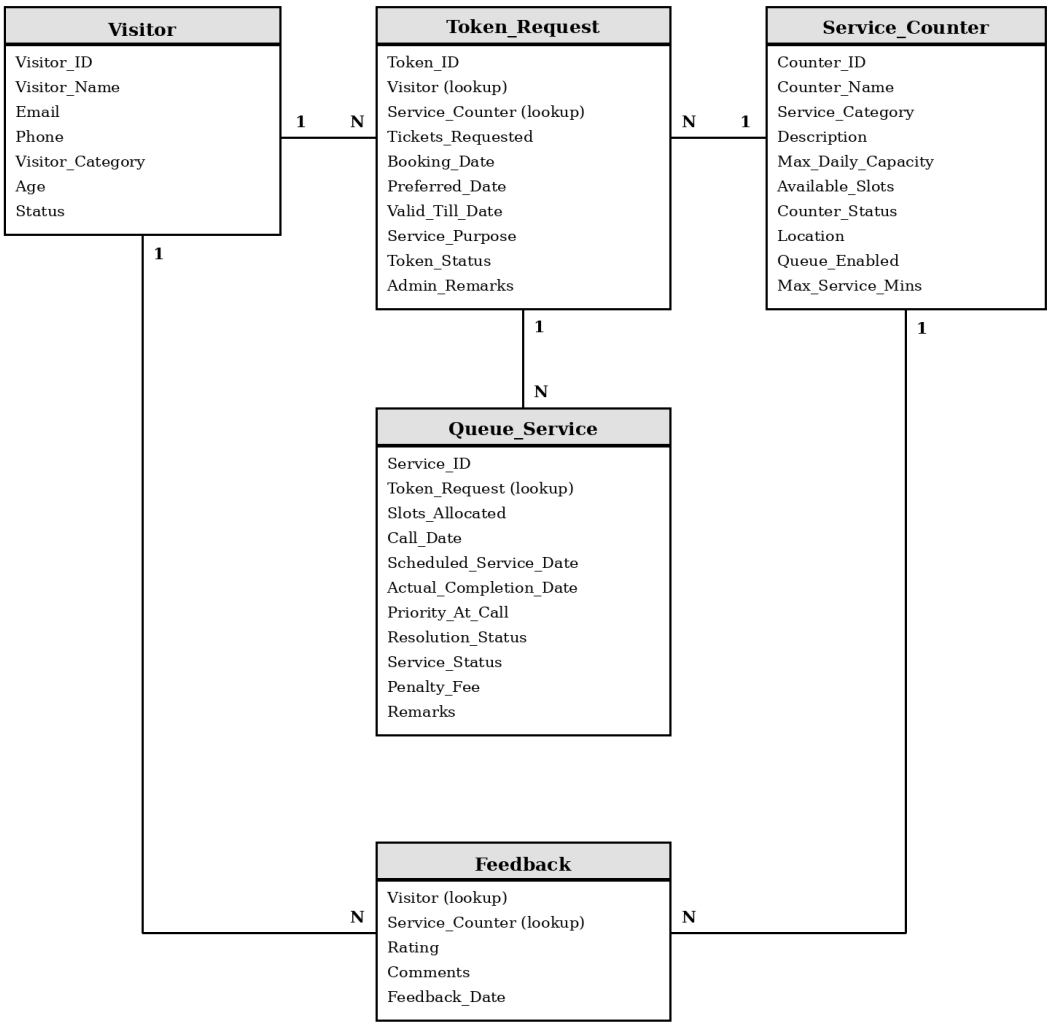


Figure 4.2: Entity–Relationship Diagram of the Application

The data model has five forms. Token_Request links a Visitor and a Service_Counter through lookup fields. Queue_Service links to Token_Request through a lookup field and does not store the visitor or counter again, because both can be reached through the linked request. Feedback links to both Visitor and Service_Counter. Table 4.1 lists the relationships.

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Visitor	Visitor_ID, Visitor_Name, Email	Token_Request	One-to-Many	Lookup field Visitor in Token_Request
Service_Counter	Counter_ID, Counter_Name, Available_Slots	Token_Request	One-to-Many	Lookup field Service_Counter in Token_Request
Token_Request	Token_ID, Tickets_Requested, Token_Status	Queue_Service	One-to-Many	Lookup field Token_Request in Queue_Service
Visitor	Visitor_ID, Visitor_Name	Feedback	One-to-Many	Lookup field Visitor in Feedback
Service_Counter	Counter_ID, Counter_Name	Feedback	One-to-Many	Lookup field Service_Counter in Feedback

4.3 Form Design

The field specification of each form is given in Tables 4.2 to 4.6. The field type shown is the Zoho Creator field type used for each field.

Table 4.2: Field Specification – Visitor Form

Field Label	Link Name	Field Type	Description / Validation
Visitor ID	Visitor_ID	Single Line	Unique visitor identifier; checked for duplicates before submission (DS-01)
Visitor Name	Visitor_Name	Single Line	Visitor's full name
Email	Email	Email	Email address used for queue notifications
Phone	Phone	Phone	Contact number
Visitor Category	Visitor_Category	Drop Down	Category of visitor (General, Student, Senior Citizen, Priority)
Age	Age	Number	Visitor's age in years
Status	Status	Drop Down	Active or Inactive

Table 4.3: Field Specification – Service_Counter Form

Field Label	Link Name	Field Type	Description / Validation
Counter ID	Counter_ID	Single Line	Unique service counter identifier
Counter Name	Counter_Name	Single Line	Name of the service counter
Service Category	Service_Category	Drop Down	Counter service category; used for the chart on the Admin Dashboard

Field Label	Link Name	Field Type	Description / Validation
Description	Description	Multi Line	Details about the services provided at the counter
Max Daily Capacity	Max_Daily_Capacity	Number	Maximum number of token slots handled per day
Available Slots	Available_Slots	Number	Slots currently available for booking; updated by call and completion workflows
Counter Status	Counter_Status	Drop Down	Open, Busy or Closed
Location	Location	Single Line	Building or floor location of the counter
Queue Enabled	Queue_Enabled	Decision Box	Indicates whether tokens can be booked for the counter
Max Service Mins	Max_Service_Mins	Number	Standard service time per token in minutes

Table 4.4: Field Specification – Token_Request Form

Field Label	Link Name	Field Type	Description / Validation
Token ID	Token_ID	Auto Number	Automatically generated token request number
Visitor	Visitor	Lookup	Lookup to the Visitor form
Service Counter	Service_Counter	Lookup	Lookup to the Service_Counter form
Tickets Requested	Tickets_Requested	Number	Must be greater than zero and not exceed the available slots (DS-02)
Booking Date	Booking_Date	Date	Date the token request is submitted
Preferred Date	Preferred_Date	Date	Preferred date of service
Valid Till Date	Valid_Till_Date	Date	Token validity deadline; must not be earlier than Preferred_Date (DS-02)
Service Purpose	Service_Purpose	Multi Line	Reason for visiting the service counter
Token Status	Token_Status	Drop Down	Pending, Approved, Rejected or Cancelled
Admin Remarks	Admin_Remarks	Multi Line	Administrator's comments and counter instructions

Table 4.5: Field Specification – Queue_Service Form

Field Label	Link Name	Field Type	Description / Validation
Service ID	Service_ID	Auto Number	Automatically generated service transaction number
Token Request	Token_Request	Lookup	Lookup to the Token_Request form; visitor and counter are reached through it
Slots Allocated	Slots_Allocated	Number	Must be greater than zero and available at the counter (DS-05)
Call Date	Call_Date	Date	Date the token is called to the counter

Field Label	Link Name	Field Type	Description / Validation
Scheduled Service Date	Scheduled_Service_Date	Date	Scheduled date for completing the service
Actual Completion Date	Actual_Completion_Date	Date	Date the visitor service was actually completed
Priority At Call	Priority_At_Call	Drop Down	Priority level when called (Normal, High, Urgent)
Resolution Status	Resolution_Status	Drop Down	Outcome at completion (Resolved, Partially Resolved, Escalated)
Service Status	Service_Status	Drop Down	Called, Served, Delayed, No-Show or Cancelled
Penalty Fee	Penalty_Fee	Number	Calculated late rescheduling fee in rupees
Remarks	Remarks	Multi Line	Additional service transaction details

Table 4.6: Field Specification – Feedback Form

Field Label	Link Name	Field Type	Description / Validation
Visitor	Visitor	Lookup	Lookup to the Visitor form
Service Counter	Service_Counter	Lookup	Lookup to the Service_Counter form
Rating	Rating	Number	Service rating from 1 to 5
Comments	Comments	Multi Line	Visitor feedback comments
Feedback Date	Feedback_Date	Date	Date feedback is recorded

4.4 Process and Workflow Design

The queue management process begins when a token request is submitted and validated, after which the administrator is notified. The administrator approves or rejects the request and the visitor is informed by email. For an approved token request, the visitor is called to the counter after validation and the available slots of the counter are reduced. The transaction remains Called until the visitor is served. If the scheduled service date passes, the daily scheduled workflow marks it Delayed and updates the rescheduling fee. On completion, the status becomes Served, the fee is calculated and the slot is restored. Feedback can be recorded after service completion.

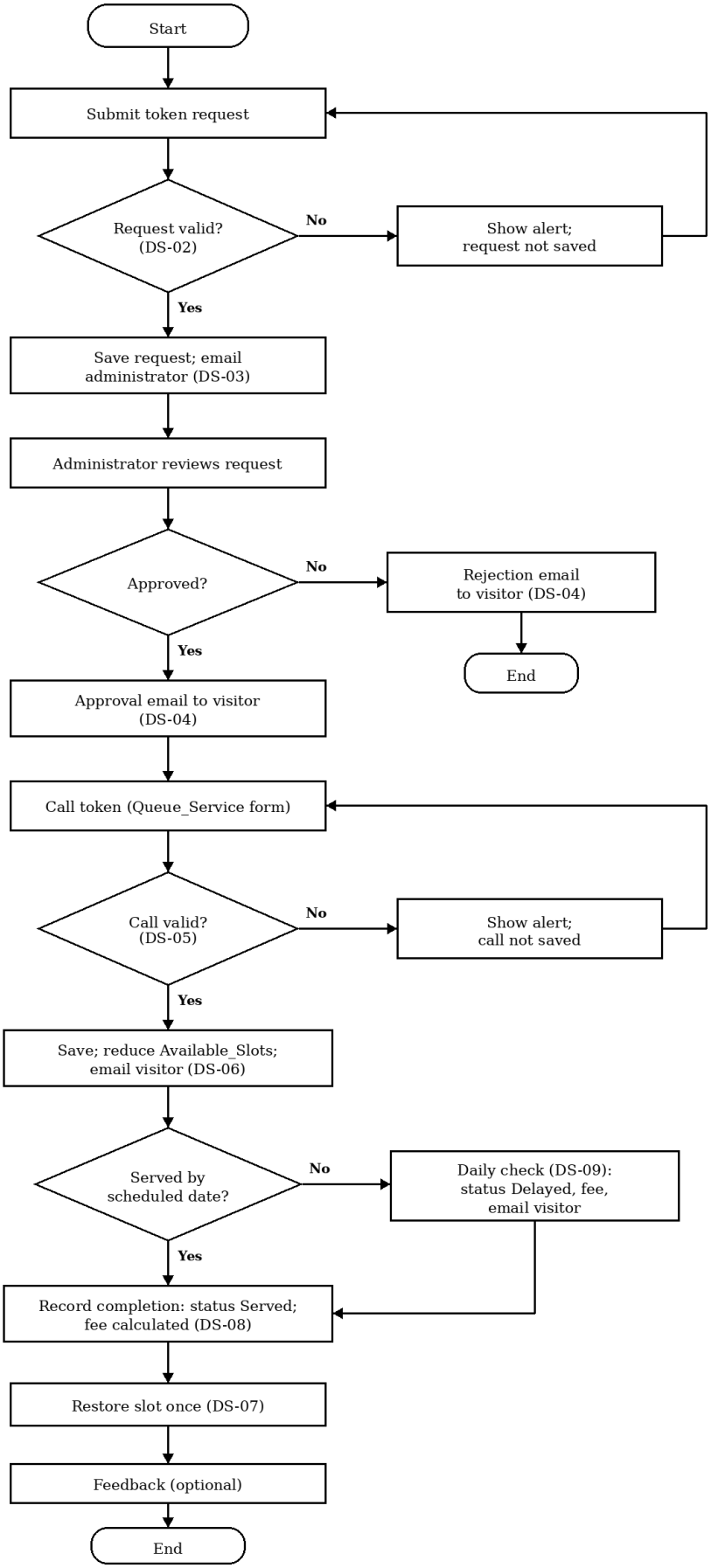

Start
Submit token request
Request valid? (DS-02)
No
Show alert; request not saved
Yes
Save request; email administrator (DS-03)
Administrator reviews request
Approved?
No
Rejection email to visitor (DS-04)
End
Yes
Approval email to visitor (DS-04)
Call token (Queue_Service form)
Call valid? (DS-05)
No
Show alert; call not saved
Yes
Save; reduce Available_Slots; email visitor (DS-06)
Served by scheduled date?
No
Daily check (DS-09): status Delayed, fee, email visitor
Yes
Record completion: status Served; fee calculated (DS-08)
Restore slot once (DS-07)
Feedback (optional)
End

Figure 4.3: Workflow / Process Flow of the Queue Management Process

4.5 User Interface and Navigation Design

The administrator opens the forms and reports from the navigation menu of the application. The Admin Dashboard is the summary page of the application. It displays the total counters, available slots, called tokens, pending requests and delayed tokens, together with a chart showing the service counters by category.

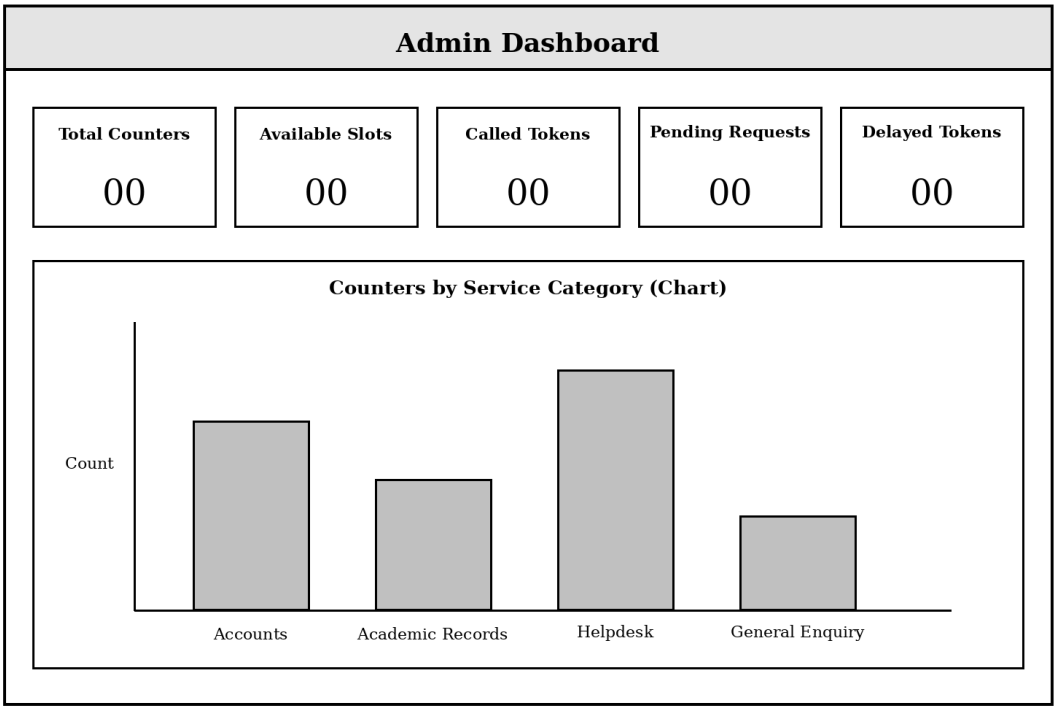


Figure 4.4: Navigation Structure and Wireframe of the Admin Dashboard

4.6 Access Control Design

The application is managed by the existing administrator account. Table 4.7 shows the permissions of the administrator. Visitors do not have a login, so they have no direct access to any form or report.

Table 4.7: Role–Permission Matrix

Component	Administrator	Visitor
Form: Visitor	A E V D	—
Form: Service_Counter	A E V D	—
Form: Token_Request	A E V D	—

Component	Administrator	Visitor
Form: Queue_Service	A E V D	—
Form: Feedback	A E V D	—
Reports (all eight reports)	V	—
Page: Admin Dashboard	V	—

A = Add, E = Edit, V = View, D = Delete, — = No access.

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application Digital Queue Management System was developed in Zoho Creator under the administrator account. It contains five forms (Visitor, Service_Counter, Token_Request, Queue_Service and Feedback), eight reports, the Admin Dashboard page and the workflows described in Section 5.6. Field link names use underscores (for example Visitor_ID and Available_Slots) so that they can be referred to easily in Deluge scripts.

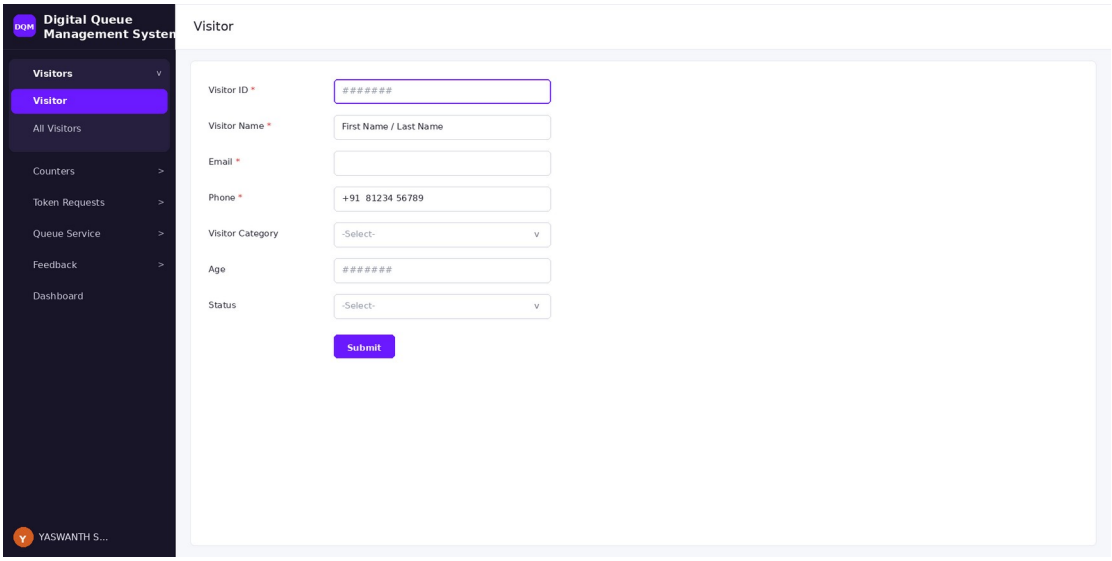


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

The Visitor form stores the visitor ID, name, email, phone, visitor category, age and status. A validation workflow checks that the Visitor_ID does not already exist before the record is saved.

The Service_Counter form stores the counter ID, name, service category, description, maximum daily capacity, available slots, counter status, location, whether queue booking is enabled and the standard service minutes. The available slots are updated automatically by the counter call and completion workflows.

The Token_Request form uses lookup fields to select the Visitor and the Service_Counter. Token_ID is generated automatically, and Token_Status holds Pending, Approved, Rejected or Cancelled. The request is validated before it is saved.

The Queue_Service form uses a lookup field to the Token_Request form. The visitor and the service counter are accessed through the linked request, which avoids duplicating data. Service_ID is generated automatically and Service_Status holds Called, Served, Delayed, No-Show or Cancelled.

The Feedback form records the visitor, service counter, rating from 1 to 5, comments and feedback date.

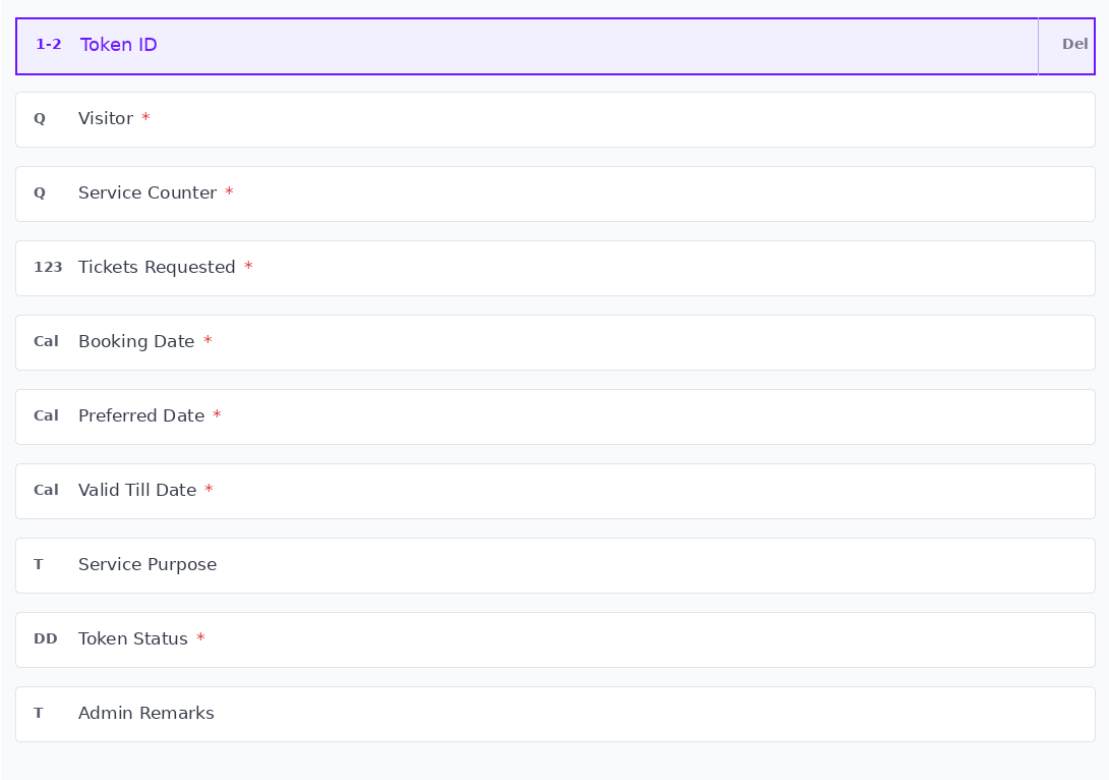


Figure 5.2: Token_Request Form in the Form Builder

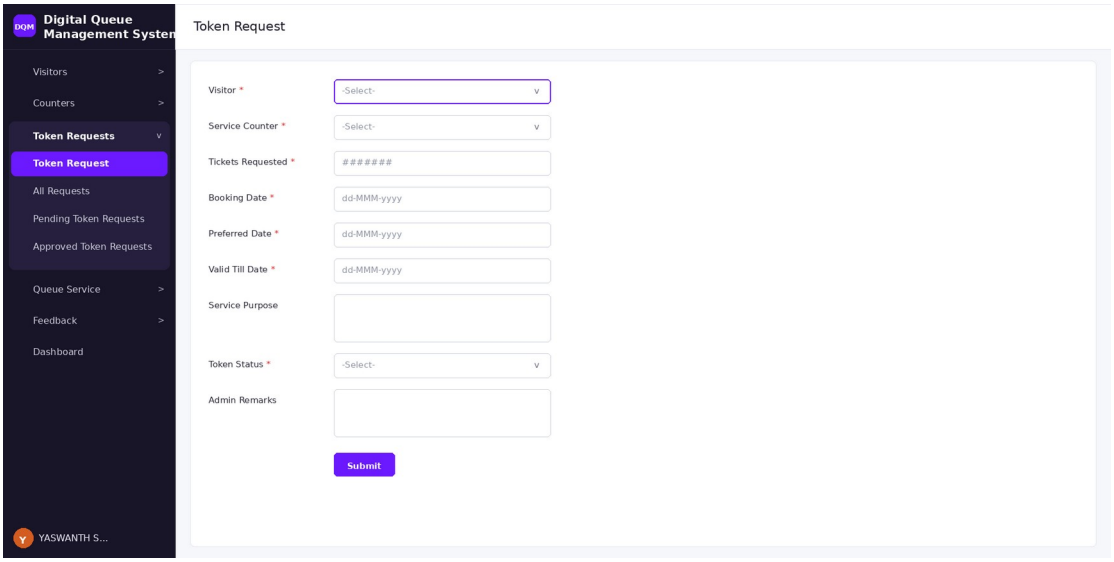


Figure 5.3: Token_Request Form in Live Mode

5.3 Reports

Eight reports were created for the administrator. They display records from the forms with the criteria given in Table 5.1 and help the administrator to monitor counter availability, review token requests and track queue service activity.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Visitors	List	Visitor	All registered visitor records	—	Administrator
Available Counters	List	Service_Counter	Available_Slots greater than zero and queue enabled	—	Administrator
Pending Token Requests	List	Token_Request	Token_Status is Pending (awaiting approval)	—	Administrator
Approved Token Requests	List	Token_Request	Token_Status is Approved	—	Administrator
Active Called Tokens	List	Queue_Service	Tokens currently called to a counter and not yet served	—	Administrator
Delayed Tokens	List	Queue_Service	Transactions whose scheduled service date has passed	—	Administrator

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Served Tokens	List	Queue_Service	Completed queue service transactions	—	Administrator
Visitor Queue History	List	Queue_Service	Service records associated with a visitor (through Token_Request)	—	Administrator

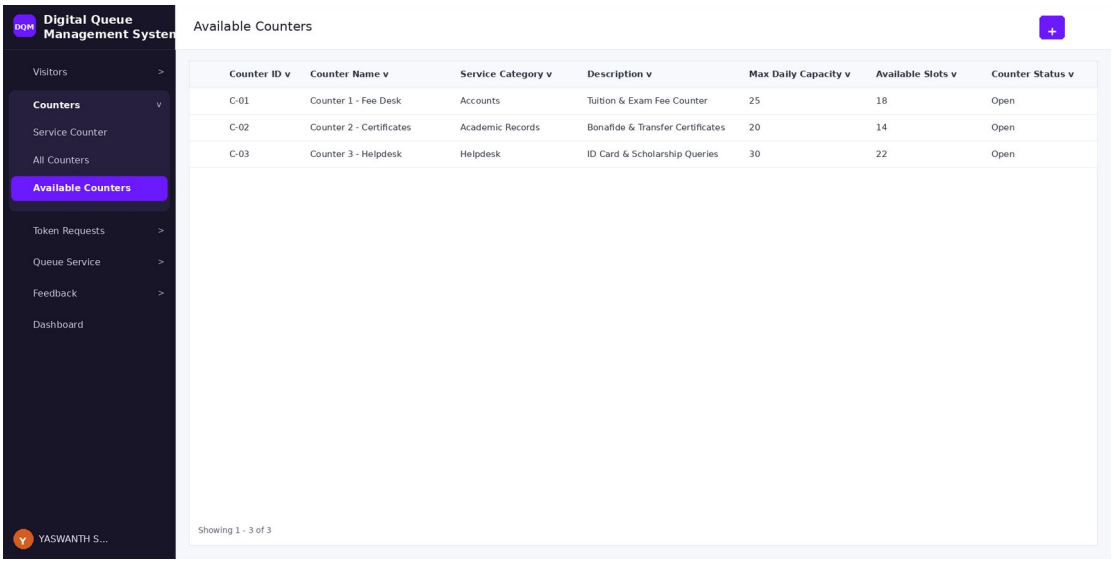


Figure 5.4: Available Counters Report

5.4 Pages and Dashboards

The Admin Dashboard is a page built for the administrator. It displays the total counters, available slots, called tokens, pending requests and delayed tokens as summary figures, and a chart showing the counters by service category. It allows the administrator to see the state of the queue management system at a glance.

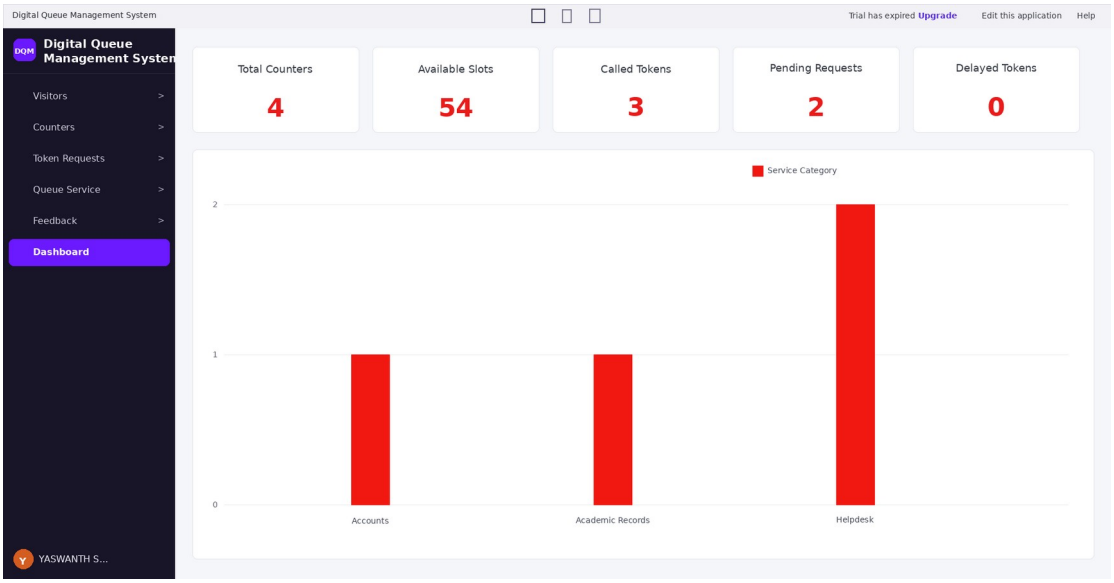


Figure 5.5: Admin Dashboard

5.5 Deluge Scripting

Deluge scripts are used in the form workflows and in the scheduled workflow. Table 5.2 lists the nine scripts. The scripts of the Token_Request validation (DS-02) and the daily delayed-token check (DS-09) are shown below, and the complete code is given in Appendix C.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Visitor → On Validate	Prevent a duplicate Visitor_ID	fetch with criteria, count(), if, alert, cancel submit
DS-02	Token_Request → On Validate	Check counter, requested tickets and dates of the request	fetch, if, alert, cancel submit
DS-03	Token_Request → On Success	Email the administrator about a new token request	sendmail, string concatenation, lookup fields
DS-04	Token_Request → On Update (status change)	Email the visitor when a token request is approved or rejected	if-else, sendmail, lookup fields
DS-05	Queue_Service → On Validate	Check the token request, slots allocated and counter capacity	fetch, if, alert, cancel submit
DS-06	Queue_Service → On Success	Reduce Available_Slots and email the visitor	fetch, field update, sendmail

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-07	Queue_Service → On Update (completion)	Set status to Served and restore the slot once	if, fetch, field update
DS-08	Queue_Service → On Completion	Calculate the late rescheduling fee at ₹10 per delayed day	date comparison, daysBetween, arithmetic
DS-09	Schedule: Daily	Mark delayed transactions, update the fee and notify the visitor	criteria fetch, for each, daysBetween, zoho.currentdate, sendmail

Code 5.1: Token request validation script (DS-02)

```
// Script ID : DS-02
// Location  : Token_Request → On Validate
// Purpose   : Check counter, requested tickets and dates of the
request

counterRec = Service_Counter[ID == input.Service_Counter];
if(counterRec.count() == 0)
{
    alert "Selected service counter does not exist.";
    cancel submit;
}
if(input.Tickets_Requested <= 0)
{
    alert "Tickets requested must be greater than zero.";
    cancel submit;
}
if(input.Tickets_Requested > counterRec.Available_Slots)
{
    alert "Requested tickets exceed the available counter slots.";
    cancel submit;
}
if(input.Valid_Till_Date < input.Preferred_Date)
{
    alert "Valid Till Date cannot be earlier than Preferred Date.";
    cancel submit;
}
```

Code 5.2: Daily delayed queue check script (DS-09)

```
// Script ID : DS-09
// Location  : Schedule: Daily
// Purpose   : Mark delayed transactions, update the fee and notify
the visitor

delayedList = Queue_Service[Service_Status == "Called" &&
Scheduled_Service_Date < zoho.currentdate];
for each rec in delayedList
{
    delayedDays =
rec.Scheduled_Service_Date.daysBetween(zoho.currentdate);
    rec.Service_Status = "Delayed";
    rec.Penalty_Fee = delayedDays * 10;
    sendmail
    [
        from :zoho.adminuserid
        to :rec.Token_Request.Visitor.Email
        subject :"Delayed Queue Token - " +
rec.Token_Request.Service_Counter.Counter_Name
        message :"Dear " + rec.Token_Request.Visitor.Visitor_Name +
",<br>Counter: " + rec.Token_Request.Service_Counter.Counter_Name +
"<br>Scheduled date: " + rec.Scheduled_Service_Date + "<br>Delayed
days: " + delayedDays + "<br>Rescheduling Fee: Rs. " + rec.Penalty_Fee
    ]
}
```

DS-02 runs when a token request is validated. It fetches the selected service counter through the lookup field and cancels the submission with an alert if the counter does not exist, the requested tickets are zero or less, the requested tickets exceed the available slots, or Valid_Till_Date is earlier than Preferred_Date. DS-09 runs daily. It fetches all transactions with the status Called whose Scheduled_Service_Date is earlier than the current date, changes the status to Delayed, sets the fee at ₹10 per delayed day and emails the visitor. Since the status is no longer Called after this change, the transaction is not processed again on later days, which prevents repeated notifications.

5.6 Workflows and Automation

The application uses nine form workflows and one scheduled workflow, listed in Table 5.3. Form workflows run on form events such as On Validate and On Success. The scheduled workflow runs once every day. Zoho approval processes, Blueprints and batch workflows were not used; approval of token requests is done by the administrator changing Token_Status.

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Visitor ID Validation	Form Workflow	Visitor – On Validate	Visitor_ID already exists	Cancel submission and show an alert	DS-01
Token Request Validation	Form Workflow	Token_Request – On Validate	Counter missing, tickets invalid or above available slots, or Valid_Till_Date earlier than Preferred_Date	Reject the request with an alert	DS-02
New Token Notification	Form Workflow	Token_Request – On Success	New token request submitted	Email the administrator	DS-03
Approval and Rejection Notification	Form Workflow	Token_Request – On Update	Token_Status changes to Approved or Rejected	Email the visitor	DS-04
Counter Call Validation	Form Workflow	Queue_Service – On Validate	Invalid token request, slots not above zero, or insufficient available slots	Cancel submission	DS-05
Token Call and Slot Update	Form Workflow	Queue_Service – On Success	Queue call transaction saved	Subtract Slots_Allocated from Available_Slots; email the visitor	DS-06
Service Completion Processing	Form Workflow	Queue_Service – On Update	Transaction changes from an active call to served (first time only)	Set status to Served; restore the slot once	DS-07
Fee Calculation	Form Workflow	Queue_Service – On Completion	Actual_Completion_Date compared with Scheduled_Service_Date	Penalty_Fee = 0 if on time, otherwise delayed days × ₹10	DS-08
Daily Delayed Check	Schedule	Daily	Status is Called and Scheduled_Service_Date is before today	Set status to Delayed; update fee; email the visitor	DS-09

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Feedback Submission	Form Workflow	Feedback – On Success	Feedback submitted	Feedback details are saved for review	—

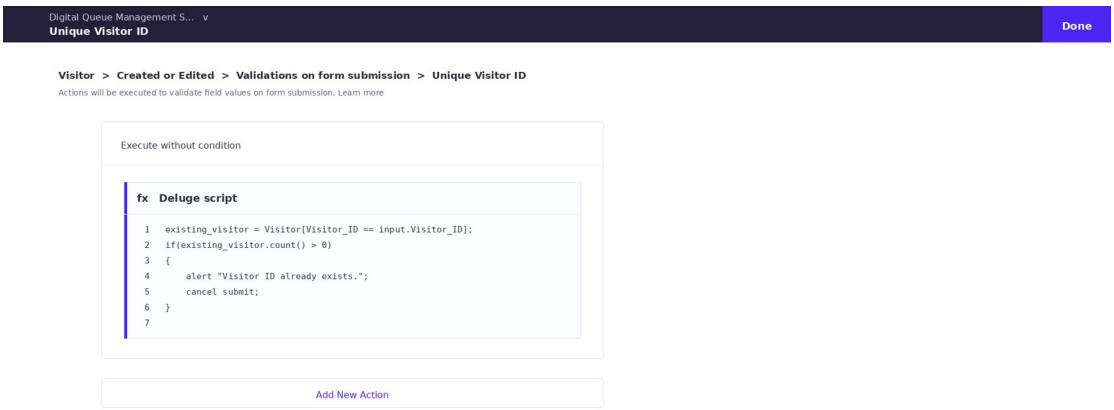


Figure 5.6: Workflow Configuration – Visitor ID Validation

5.7 Integrations

No external API, webhook or other Zoho service is integrated with this application. The only integration used is the email notification through the Deluge sendmail task, as shown in Table 5.4. Integration with other systems is listed as a future enhancement.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Email Notifications	Zoho Creator email (sendmail task)	Deluge sendmail in form and scheduled workflows	No separate authentication required	Emails to the administrator and visitors: new token request, approval or rejection, token called to counter and delayed notice

5.8 User Management and Role-Based Access

The application uses the existing administrator account to manage the system. The administrator maintains visitor and service counter records, reviews token requests, approves or rejects bookings, manages counter call and completion transactions and views the reports. Separate visitor login roles and a customer portal are not implemented because the current Zoho Creator plan requires an upgrade for the required access features. Therefore, record-level access rules and permission profiles for other roles were not configured, and visitor activities are demonstrated through sample records managed within the application.

5.9 AI and Analytics Features

No AI features such as Zia were used in this application. For analytics, the Admin Dashboard displays summary figures (total counters, available slots, called tokens, pending requests and delayed tokens) and a chart of counters by service category, as shown in Figure 5.5. These give the administrator a quick view of the counter capacities and queue status.

5.10 Pro-Code Extension

A pro-code extension was not used because all the required logic – validation, counter slot update, fee calculation, delayed-queue checking and email notification – could be written in Deluge within the form and scheduled workflows. If the application grows, Zoho Catalyst or the Creator REST API could be used to connect it with external kiosk displays or SMS gateways.

5.11 Mobile and Portal Access

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

The application was tested at the level of individual forms and scripts and then through the complete queue management process. Each form was tested by submitting valid and invalid data and checking the alerts and the saved records. The workflows were tested by following a token request from submission to approval, counter call and completion, and by checking the available slots, status, fee and emails at each step. The delayed-queue workflow was tested with a transaction whose scheduled service date had passed. The reports and the Admin Dashboard were checked against the records in the forms. Since the application is used through the administrator account, role-based access testing was not applicable.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Add a visitor with a new Visitor_ID	Enter valid visitor details and submit	Record is saved	Visitor record saved successfully.	Pass
TC-02	FR-01	Add a visitor with an existing Visitor_ID	Enter an existing Visitor_ID and submit	Alert is shown and submission is cancelled	Alert shown for duplicate Visitor_ID; submission cancelled.	Pass
TC-03	FR-02	Add a service counter	Enter counter details with max capacity and available slots	Record is saved	Service counter record saved successfully.	Pass
TC-04	FR-03	Submit a valid token request	Select visitor and counter; tickets within available slots; valid dates	Request is saved with an automatic Token_ID	Token request saved with automatic Token_ID.	Pass
TC-05	FR-03	Tickets requested is zero	Enter 0 as Tickets_Requested	Alert is shown and request is not saved	Alert shown; request not saved.	Pass
TC-06	FR-03	Tickets requested exceeds available slots	Enter a count above Available_Slots	Alert is shown and request is not saved	Alert shown; request not saved.	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-07	FR-03	Valid_Till_Date earlier than Preferred_Date	Enter an earlier Valid_Till_Date	Alert is shown and request is not saved	Alert shown; request not saved.	Pass
TC-08	FR-04	Administrator notification	Submit a valid token request	Email with visitor, counter, tickets and dates is sent to the administrator	Email with request details received by the administrator.	Pass
TC-09	FR-05	Approve a token request	Change Token_Status to Approved	Approval email is sent to the visitor	Approval email received by the visitor.	Pass
TC-10	FR-05	Reject a token request	Change Token_Status to Rejected with remarks	Rejection email is sent to the visitor	Rejection email with remarks received by the visitor.	Pass
TC-11	FR-06	Allocate more than available slots	Enter Slots_Allocated above Available_Slots	Submission is cancelled	Submission cancelled with an alert.	Pass
TC-12	FR-07	Call a token to a counter	Enter a valid queue service call transaction	Available_Slots is reduced and the visitor is emailed	Available_Slots reduced; call email received by the visitor.	Pass
TC-13	FR-08, FR-09	Complete service on time	Enter Actual_Completion_Date on or before the scheduled date	Status is Served, fee is 0 and slot is restored once	Status set to Served, fee 0, slot restored once.	Pass
TC-14	FR-09	Complete service late	Enter Actual_Completion_Date three days after the scheduled date	Penalty_Fee is ₹30 and status is Served	Fee calculated as ₹30; status set to Served.	Pass
TC-15	FR-10	Daily delayed queue check	Run the check with a Called transaction past its scheduled date	Status is Delayed, fee is updated and the visitor is emailed once	Status changed to Delayed, fee updated and email sent once.	Pass
TC-16	FR-11	Submit feedback	Select visitor and counter; enter rating and comments	Feedback record is saved	Feedback record saved successfully.	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-17	FR-12, FR-13	View reports and dashboard	Open the reports and the Admin Dashboard	Records and counts match the stored data	Reports and dashboard counts matched the stored data.	Pass

6.3 Debugging Deluge Scripts

Scripts were debugged by submitting forms with valid and invalid data and observing the alerts and the changes in the saved records. Error messages shown by the Deluge editor were used to correct syntax errors, and the available slots, status, fee and emails were checked after each workflow ran. Problems that could cause repeated emails or repeated slot updates were identified and handled in the workflow conditions, as listed in Table 6.2.

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Approval or rejection email could be sent again whenever a token request was edited for another reason	Workflow ran on every edit of the request	Notification runs only when Token_Status changes to Approved or Rejected	Closed
BUG-02	Available slots could increase more than once for the same completed service	Completion logic could run on every edit of the transaction	Slots are restored only when the status first changes from an active call to Served	Closed
BUG-03	Delayed token emails could be repeated every day	The scheduled workflow could pick up the same transaction again	Only transactions with status Called are processed; status becomes Delayed after the first run	Closed
BUG-04	Counter slots could be reduced even if the call transaction failed	Slot update ran before the record was saved	Slot update is done in On Success after the record is saved	Closed
BUG-05	Duplicate visitor records could be created	No check on Visitor_ID	Duplicate Visitor_ID check in On Validate (DS-01)	Closed

6.5 User Acceptance Testing

The application was tried by two users, and their feedback and the action taken are recorded in Table 6.3.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Arun K	Administrator (sample visitor flow)	The token request form is easy to use and the slot capacity alerts are clear.	No change required.
Meena R	Administrator (counter and completion handling)	Late fee and delayed-token emails are useful; the dashboard gives a clear summary.	Dashboard layout reviewed and kept as it is; counts verified against records.

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application is developed and used within a single Zoho Creator environment under the administrator account. A separate Stage and Production arrangement was not used in this project. The application is managed by the administrator, and visitor activities are demonstrated through sample records. The deployment checklist of the application is given in Table 7.1.

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✔)
1	All test cases executed in Development / Stage	✓
2	Sample/test data removed or isolated	N/A
3	Roles, permissions and users configured in Production	✓
4	Connections authorised in Production	N/A
5	Schedules and workflows enabled	✓
6	Application published and URL verified	✓
7	Version / release notes recorded	✓

7.2 Performance Optimisation

The following practices were followed to keep the application efficient. Reports use criteria so that only the required records are displayed, for example the Available Counters report shows only counters with available slots above zero and queue booking enabled. The Queue_Service form reads the visitor and the counter through the Token_Request lookup instead of storing them again. Fetch statements use criteria so that only the required record is retrieved. The scheduled workflow selects only transactions with the status Called, so transactions that are already Delayed are skipped on later days. The counter slot count is updated only after the transaction is saved successfully. Since the application is demonstrated with sample records, formal performance measurements are not reported.

7.3 Security Features

Security in this application is based on administrator-only access and validation of the data entered. No other user accounts exist, so only the administrator can view or change the records. Table 7.2 lists the controls.

Table 7.2: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Administrator-only access	Whole application	Unauthorised viewing or modification of queue records
Validation workflows (DS-01, DS-02, DS-05)	Visitor, Token_Request and Queue_Service forms	Duplicate or invalid data and allocating more than the available counter slots
Controlled slot updates (DS-06, DS-07)	Counter call and service completion workflows	Incorrect counter slot figures caused by repeated updates

7.4 Governance and Administration

The application is maintained by the administrator, who is responsible for keeping the visitor and service counter records correct and for processing token requests and service transactions. Field and form names follow a common naming style with underscores, which makes the application and its scripts easy to read. Every queue service transaction keeps its dates, status, priority, resolution status, fee and remarks in the Queue_Service form, and the administrator's comments on a request are kept in Admin_Remarks, so the history of queue activity can be reviewed at any time.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The working of the application is shown with one queue scenario. A visitor requests a queue token, the administrator reviews the request and calls the token to the counter, and the service is later completed. In Step 1, the token request is submitted and validated, and the administrator is notified by email. In Step 2, the administrator approves the request and calls the token, and the available slots of the counter are reduced. In Step 3, the service is completed, the status is changed to Served, the fee is calculated and the counter slot is restored. The reports and the Admin Dashboard reflect these changes.

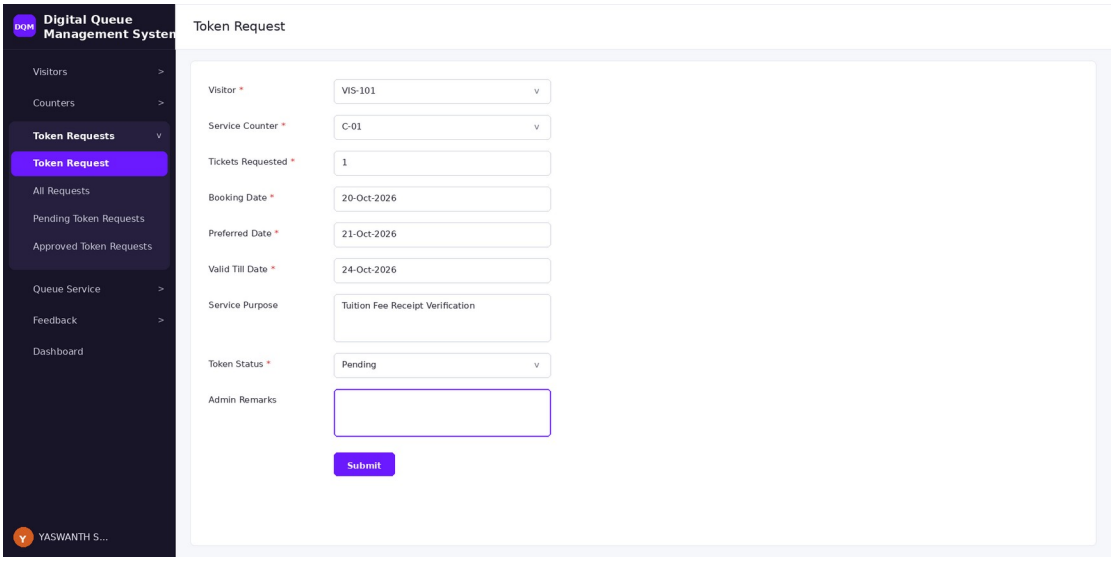


Figure 8.1: Scenario Step 1 – Token Request Submitted

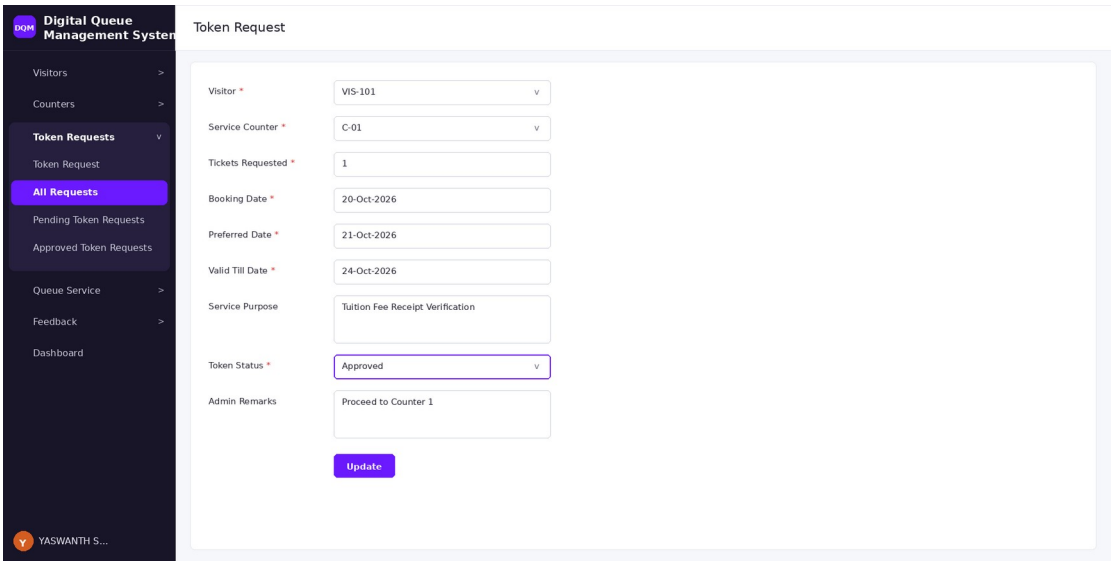


Figure 8.2: Scenario Step 2 – Token Approved and Called to Counter

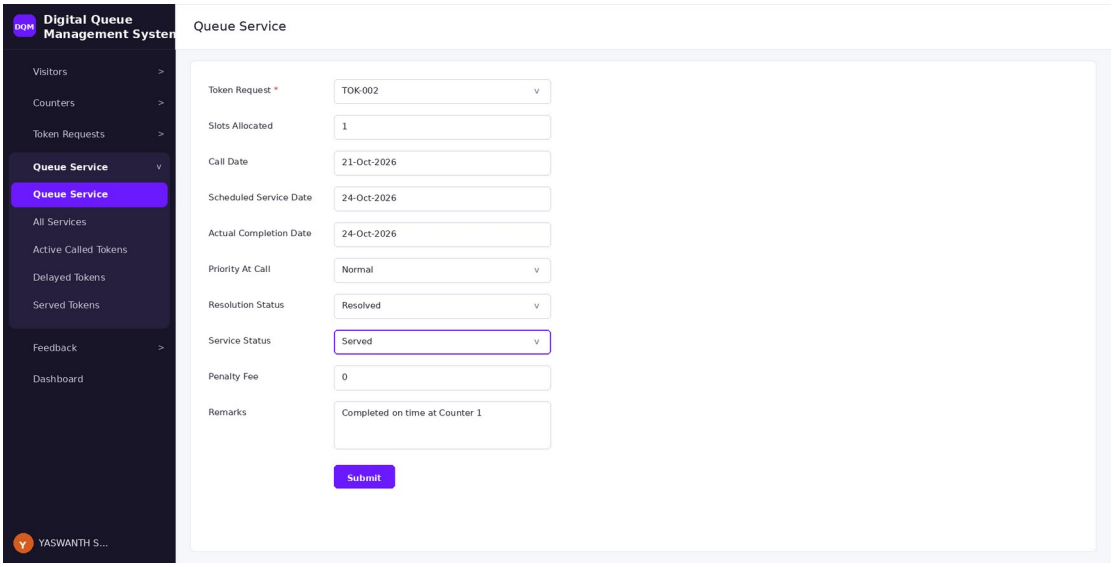


Figure 8.3: Scenario Step 3 – Service Completed and Fee Calculated

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
To design a data model and forms	Sections 4.2, 4.3 and 5.2	Five forms connected through lookup fields
To automate validation, slot update, service completion, fee and delay checking	Sections 5.5 and 5.6	Nine Deluge scripts in nine form workflows and one scheduled workflow
To provide reports and an Admin Dashboard	Sections 5.3 and 5.4	Eight reports and the Admin Dashboard with a service category chart
To send automated email notifications	Sections 5.5 to 5.7	Emails for new token requests, approval or rejection, called tokens and delayed tokens
To demonstrate the complete queue management process	Section 8.1	Scenario from token request to service completion shown in Figures 8.1 to 8.3

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Checking counter capacity	Available slots are checked by hand in a register	Counter slots are checked automatically when a token request or call is entered	Prevents over-booking of counters
Manual steps	Slot update, fee calculation and delay checking are done by hand	Done automatically by workflows	Fewer manual steps
Notifications	No automatic notifications	Email notifications to the administrator and visitors	Timely queue information
Report generation	Lists are prepared by hand	Ready-made reports and the Admin Dashboard	Quick view of queue activity
Data accessibility	Records are scattered in registers or spreadsheets	All records are stored together in Zoho Creator	Single source of data

8.4 Limitations of the Low-Code Approach

Some limitations were met in this project. The current Zoho Creator plan requires an upgrade for separate visitor login roles and a customer portal, so visitor activities are demonstrated through sample records managed by the administrator. Slots_Allocated should not be changed freely after a token is called, because the counter capacity is adjusted only once at the time of calling; changing it later would need additional adjustment logic. For no-show or cancelled tokens, the administrator records the status and adjusts the counter slots according to the situation rather than the slot being restored automatically. Customisation is also limited to the features of Zoho Creator and Deluge [6].

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

On the spectrum from no-code to pro-code, most of the application is no-code configuration: the forms, lookup fields, reports and the Admin Dashboard were created visually. The validations, counter slot updates, fee calculation, delayed-token checking and email notifications needed low-code Deluge scripting. No pro-code

development was used. A move to pro-code would be considered only if the application needs to connect with external hardware display boards or needs processing that Deluge cannot handle conveniently.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Digital Queue Management System was developed on Zoho Creator to manage visitor token requests and counter service operations. The application has five forms for visitors, service counters, token requests, queue service transactions and feedback; eight reports and an Admin Dashboard; and Deluge workflows that validate data, update counter slots, calculate rescheduling fees at ₹10 per delayed day, check delayed tokens daily and send email notifications.

The application demonstrates counter capacity tracking, token request validation, administrative approval, counter call and completion management, delay monitoring, fee calculation and email-based communication in a centralised system. It is managed through the administrator account, and visitor activities are demonstrated through sample records because the current plan does not provide visitor login access.

9.2 Future Enhancements

- Visitor login roles and a self-service portal, so that visitors can book queue tokens and track their queue position themselves.
- Additional adjustment logic for changes to Slots_Allocated after calling and for no-show or cancelled tokens.
- Automated SMS and email reminders before the scheduled service date in addition to the delayed notice.
- Further reports and analytics on counter waiting times, peak service hours and visitor feedback ratings.
- Integration with live digital display boards and campus ERP systems through the Creator REST API or Zoho Catalyst.

9.3 Learning Outcomes and Career Relevance

Working on this project gave practical experience in designing forms and lookup relationships, writing Deluge scripts for validation and automation, building scheduled workflows, creating reports and a dashboard and sending automated emails. These skills are relevant to roles such as low-code developer, citizen developer, solutions consultant and automation engineer, where business processes are converted into working applications quickly.

REFERENCES

[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 20 October 2026).

[3] Zoho Corporation, “Deluge Script Reference Guide.” [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 20 October 2026).

[4] A. C. Bock and U. Frank, “Low-code platform,” *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733–740, 2021.

[5] A. Sahay, A. Indamutsa, D. Di Ruscio, and A. Pierantonio, “Supporting the understanding and comparison of low-code development platforms,” in *Proc. 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA)*, 2020, pp. 171–178.

[6] Y. Luo, P. Liang, C. Wang, M. Shahin, and J. Zhan, “Characteristics and challenges of low-code development: The practitioners’ perspective,” in *Proc. ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM)*, 2021.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Digital Queue Management System application and its navigation	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	Y	Visitor, Service_Counter, Token_Request, Queue_Service and Feedback forms	5.2	Tables 4.2–4.6
II	Lookup field (relationship between forms)	Y	Visitor and Service_Counter lookups in Token_Request, Token_Request lookup in Queue_Service, Visitor and Service_Counter lookups in Feedback	4.2, 5.2	Fig. 4.2, Table 4.1
II	Subform (parent–child data)	N	Not used	4.2, 5.2	—
II	Field validations / layout rules	Y	On Validate workflows (DS-01, DS-02, DS-05)	5.2	Code 5.1
II	List report with filter, grouping, custom action	Y	Eight filtered list reports	5.3	Table 5.1, Fig. 5.4
II	Kanban / Calendar / Timeline / Map report	N	Not used	5.3	—
II	Pivot table / chart report	N	Not used as a report; the service category chart is on the Admin Dashboard	5.3	—
II	Page / dashboard built with UI Builder	Y	Admin Dashboard page	5.4	Fig. 5.5

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Deluge – form events (validate / success / field actions)	Y	Form workflows on Visitor, Token_Request, Queue_Service and Feedback	5.5	Table 5.3, Appendix C
II	Deluge – functions, fetch, loops, collections	Y	Criteria fetch and for each loop (DS-09); fetch in DS-01 to DS-08	5.5	Code 5.2, Appendix C
II	Email / in-app notifications	Y	Email notifications (DS-03, DS-04, DS-06, DS-09)	5.5, 5.6	Appendix C
II	Form workflow	Y	Nine form workflows	5.6	Table 5.3
II	Scheduled workflow	Y	Daily Delayed Check	5.6	Code 5.2, Fig. 5.6
II	Approval process	N	Not used; approval is done by changing Token_Status	5.6	—
II	Blueprint	N	Not used	5.6	—
II	Batch / report workflow	N	Not used	5.6	—
II	Testing & debugging evidence	Y	Test cases and defect log	6	Tables 6.1, 6.2
III	Integration with a Zoho service	N	No Zoho service integration	5.7	—
III	Connection / REST API call (invokeurl)	N	Not used	5.7	—
III	Webhook or Creator REST API	N	Not used	5.7	—
III	Users, roles and permission profiles	N	Only the existing administrator account is used	5.8	—
III	Record-level access criteria / customer portal	N	Not implemented	5.8	—
III	Zia / AI feature	N	Not used	5.9	—
III	Analytics (charts / Zoho Analytics)	Y	Summary figures and service category chart on the Admin Dashboard	5.9	Fig. 5.5

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
IV	Performance optimisation applied & measured	N	Techniques described but not measured	7.2	—
IV	Deployment across environments	N	Single environment used	7.1	—
IV	Security controls (encryption, OAuth, permissions)	Y	Administrator-only access and validation workflows	7.3	Table 7.2
IV	Governance (audit trail, administration)	N	Not implemented	7.4	—
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used	5.10	—
V	Low-code limitations & spectrum analysis	Y	Discussed in Sections 8.4 and 8.5	8.4, 8.5	—
—	Mobile app / portal access	N	Not implemented	5.11	—

Total features implemented: 15 / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	High	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	High	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	Medium	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	High	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	Medium	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Visitor – Visitor ID Validation

```
// Script ID : DS-01
// Location  : Visitor → On Validate
// Purpose   : Prevent a duplicate Visitor_ID

dup = Visitor[Visitor_ID == input.Visitor_ID];
if(dup.count() > 0)
{
    alert "Visitor ID already exists. Please enter a unique Visitor
ID.";
    cancel submit;
}
```

C.2 Token_Request – Token Request Validation

```
// Script ID : DS-02
// Location  : Token_Request → On Validate
// Purpose   : Check counter, requested tickets and dates of the
request

counterRec = Service_Counter[ID == input.Service_Counter];
if(counterRec.count() == 0)
{
    alert "Selected service counter does not exist.";
    cancel submit;
}
if(input.Tickets_Requested <= 0)
{
    alert "Tickets requested must be greater than zero.";
    cancel submit;
}
if(input.Tickets_Requested > counterRec.Available_Slots)
{
    alert "Requested tickets exceed the available counter slots.";
    cancel submit;
}
if(input.Valid_Till_Date < input.Preferred_Date)
{
    alert "Valid Till Date cannot be earlier than Preferred Date.";
    cancel submit;
}
```

C.3 Token_Request – New Token Notification

```
// Script ID : DS-03
// Location  : Token_Request → On Success
// Purpose   : Email the administrator about a new token request

sendmail
[
    from :zoho.adminuserid
    to :zoho.adminuserid
    subject :"New Queue Token Request " + input.Token_ID
    message :"A new queue token request has been
submitted.<br>Visitor: " + input.Visitor.Visitor_Name + "<br>Counter:
" + input.Service_Counter.Counter_Name + "<br>Tickets: " +
input.Tickets_Requested + "<br>Preferred Date: " +
input.Preferred_Date + "<br>Valid Till: " + input.Valid_Till_Date +
"<br>Please review the token request."
]
```

C.4 Token_Request – Approval and Rejection Notification

```
// Script ID : DS-04
// Location  : Token_Request → On Update (status change)
// Purpose   : Email the visitor when a token request is approved or
rejected

// Executes only when Token_Status is updated
if(input.Token_Status == "Approved")
{
    sendmail
    [
        from :zoho.adminuserid
        to :input.Visitor.Email
        subject :"Queue Token Approved - " + input.Token_ID
        message :"Dear " + input.Visitor.Visitor_Name + ",<br>Your
token request for " + input.Service_Counter.Counter_Name + " has been
approved. Please report to the counter as scheduled.<br>Remarks: " +
ifnull(input.Admin_Remarks,"")
    ]
}
else if(input.Token_Status == "Rejected")
{
    sendmail
    [
        from :zoho.adminuserid
        to :input.Visitor.Email
        subject :"Queue Token Rejected - " + input.Token_ID
        message :"Dear " + input.Visitor.Visitor_Name + ",<br>Your
token request for " + input.Service_Counter.Counter_Name + " has been
rejected.<br>Remarks: " + ifnull(input.Admin_Remarks,"")
    ]
}
```

C.5 Queue_Service – Counter Call Validation

```
// Script ID : DS-05
// Location  : Queue_Service → On Validate
// Purpose   : Check the token request, slots allocated and counter
capacity

if(input.Token_Request == null)
{
    alert "Please select a valid token request.";
    cancel submit;
}
if(input.Slots_Allocated <= 0)
{
    alert "Slots allocated must be greater than zero.";
    cancel submit;
}
counterRec = Service_Counter[ID ==
input.Token_Request.Service_Counter];
if(input.Slots_Allocated > counterRec.Available_Slots)
{
    alert "Insufficient counter slots available for calling.";
    cancel submit;
}
```

C.6 Queue_Service – Token Call and Slot Update

```
// Script ID : DS-06
// Location  : Queue_Service → On Success
// Purpose   : Reduce Available_Slots and email the visitor

reqRec = Token_Request[ID == input.Token_Request];
counterRec = Service_Counter[ID == reqRec.Service_Counter];
counterRec.Available_Slots = counterRec.Available_Slots -
input.Slots_Allocated;
sendmail
[
    from :zoho.adminuserid
    to :reqRec.Visitor.Email
    subject :"Token Called to Counter - " + counterRec.Counter_Name
    message :"Dear " + reqRec.Visitor.Visitor_Name + ",<br>" +
input.Slots_Allocated + " slot(s) at " + counterRec.Counter_Name + "
have been called for your service.<br>Scheduled service date: " +
input.Scheduled_Service_Date
]
```

C.7 Queue_Service – Service Completion Processing

```
// Script ID : DS-07
// Location  : Queue_Service → On Update (completion)
// Purpose   : Set status to Served and restore the slot once

// Process only the first completion (Called or Delayed -> Served)
if(input.Actual_Completion_Date != null && (input.Service_Status ==
"Called" || input.Service_Status == "Delayed"))
{
    reqRec = Token_Request[ID == input.Token_Request];
    counterRec = Service_Counter[ID == reqRec.Service_Counter];
    counterRec.Available_Slots = counterRec.Available_Slots +
input.Slots_Allocated;
    input.Service_Status = "Served";
}
```

C.8 Queue_Service – Fee Calculation

```
// Script ID : DS-08
// Location  : Queue_Service → On Completion
// Purpose   : Calculate the rescheduling fee at ₹10 per delayed day

if(input.Actual_Completion_Date <= input.Scheduled_Service_Date)
{
    input.Penalty_Fee = 0;
}
else
{
    delayedDays =
input.Scheduled_Service_Date.daysBetween(input.Actual_Completion_Date)
;
    input.Penalty_Fee = delayedDays * 10;
}
```

C.9 Scheduled Workflow – Daily Delayed Check

```
// Script ID : DS-09
// Location  : Schedule: Daily
// Purpose   : Mark delayed transactions, update the fee and notify
the visitor

delayedList = Queue_Service[Service_Status == "Called" &&
Scheduled_Service_Date < zoho.currentdate];
for each rec in delayedList
{
    delayedDays =
rec.Scheduled_Service_Date.daysBetween(zoho.currentdate);
    rec.Service_Status = "Delayed";
    rec.Penalty_Fee = delayedDays * 10;
    sendmail
    [
        from :zoho.adminuserid
        to :rec.Token_Request.Visitor.Email
        subject :"Delayed Queue Token - " +
rec.Token_Request.Service_Counter.Counter_Name
        message :"Dear " + rec.Token_Request.Visitor.Visitor_Name +
",<br>Counter: " + rec.Token_Request.Service_Counter.Counter_Name +
"<br>Scheduled date: " + rec.Scheduled_Service_Date + "<br>Delayed
days: " + delayedDays + "<br>Rescheduling Fee: Rs. " + rec.Penalty_Fee
    ]
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the Zoho Creator address of the application in a web browser and sign in with the administrator account. The application opens with its navigation menu, from which the forms, reports and the Admin Dashboard can be opened. Visitors do not log in to the application.

D.2 Instructions for Administrator

- Open the Visitor form to add a visitor. Enter a unique Visitor_ID; a duplicate ID is rejected with an alert.
- Open the Service_Counter form to add a counter with its maximum daily capacity, available slots, status, location, queue permission and standard service minutes.
- Open the Token_Request form, select the visitor and the service counter, and enter the tickets requested, dates and service purpose. Correct the entry if an alert is shown.
- Open the Pending Token Requests report, review each request and change Token_Status to Approved or Rejected; add Admin_Remarks if required. The visitor is emailed automatically.
- Open the Queue_Service form to call an approved token to a counter. Select the Token_Request, enter the slots allocated, call date, scheduled service date and priority at call. The available slots are reduced automatically.
- When the visitor is served, edit the transaction and enter Actual_Completion_Date and Resolution_Status. The status changes to Served, the fee is calculated and the slot is restored.
- For a no-show or cancelled token, record the appropriate status and adjust the counter capacity according to the situation.
- Use the Active Called Tokens, Delayed Tokens, Served Tokens, Approved Token Requests, Available Counters, All Visitors and Visitor Queue History reports to monitor queue activity.

- Open the Admin Dashboard to see the totals and the chart of counters by service category.
- Open the Feedback form to record the visitor's rating and comments for a completed counter service.

D.3 Instructions for Visitor

- Visitors do not have a login in the current application; their activities are recorded by the administrator.
- The visitor receives an email when a token request is approved or rejected, and when a token is called to a counter with its scheduled service date.
- Report to the counter on or before the scheduled service date. A late completion carries a rescheduling fee of ₹10 per delayed day.
- If a called token becomes delayed, the visitor receives an email showing the counter, the scheduled date, the delayed days and the fee.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	YASWANTH SREENIVAS D	Register No.	240701611
Project Title	Digital Queue Management System	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner **Signature of External Examiner**

Name: _____ Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	YASWANTH SREENIVAS D	Register No.	240701611
Department / Section	CSE	Date of Viva	
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____